CLOZFLOW COGNITIVE BRAIN SPECIFICATION

Volume X

Implementation Engineering Specification

The complete, build-ready technical specification for implementing the Clozflow Cognitive Brain from source — covering data contracts, API interfaces, technology stack, latency budgets, infrastructure, data schemas, deployment, testing, and onboarding sequences.

Document ID	CCBS-V10-IMPL
Version	1.0
Status	Definitive — Implementation Reference
Volume	X — Implementation Engineering Specification
Depends on	Volumes I – IX (all chapters)
Audience	AI Engineers · Backend Engineers · ML Engineers · DevOps · Platform Architects
Purpose	Provides every concrete technical detail required for any engineering team or AI system to implement the Clozflow Brain end-to-end, without ambiguity.
Date	June 28, 2026

After completing all ten volumes, any competent AI or engineering team possesses the full specification required to build, test, deploy, and operate the Clozflow Cognitive Brain in production.

How to Use This Volume

Volumes I–IX established the philosophy, architecture, cognitive theory, execution models, runtime design, extensibility frameworks, enterprise integration, engineering methodology, and assessment architecture of the Clozflow Brain. They answer: what the Brain is, why it is designed this way, and what it must do.

Volume X answers: how to build it. Every section resolves a specific implementation gap identified in the prior volumes — translating architectural intent into concrete, unambiguous engineering specifications.

Reading Order for Implementors

- Read Volumes I–IV first to understand the cognitive model before touching implementation.
- Use Volume V (COS) alongside Part II of this volume — they describe the same runtime from different angles.
- Parts III–V are self-contained — engineers can work on stack, infrastructure, and testing in parallel.
- Part VI (Onboarding Sequence) provides the recommended build order for a team starting from scratch.

PART I

Data Contracts, Schemas & Event Specifications

All cognitive processing in the Brain operates on structured data objects. This part defines every canonical data structure: the Cognitive Event envelope, the World Model state object, the Cognitive Reality Model node, the Recommendation output, and all intermediate structures exchanged between modules.

Chapter 1: Cognitive Event Schema

1.1 Event Envelope — Canonical JSON Schema

Every piece of information entering the Brain must be transformed into a Cognitive Event before processing. The following schema is the authoritative definition. All producers must validate against it before publishing to the Event Bus.

```
{
"event_id": "string — UUID v4, globally unique",
"event_type": "string — see EventType registry §1.2",
"session_id": "string — UUID v4, ties event to a Brain Session",
"tenant_id": "string — UUID v4, enterprise tenant identifier",
"timestamp_utc": "string — ISO 8601, e.g. 2025-06-01T14:32:00.123Z",
"sequence_num": "integer — monotonic within session, for ordering",
"source": "string — producing subsystem identifier",
"priority": "integer — 1 (critical) to 5 (background)",
"ttl_ms": "integer — event expires after N ms if unprocessed",
"payload": "object — type-specific, see §1.3",
"confidence": "float — 0.0–1.0, producer confidence in event accuracy",
"causation_id": "string? — event_id that caused this event (chain tracing)",
"correlation_id": "string? — groups related events across a reasoning cycle",
"metadata": "object? — optional key-value pairs for observability"
}
```

1.2 EventType Registry

The following table defines every recognised event type, its triggering condition, priority ceiling, and expected TTL. Engineers must not introduce new types without updating this registry and the Cognitive Event Router (§2.4).

EventType	Trigger Condition	Priority	TTL (ms)	Payload Schema
TRANSCRIPT_SEGMENT	New speech-to-text chunk arrives	1	200	TranscriptPayload §1.3.1
SILENCE_DETECTED	No speech for > threshold_ms	2	500	SilencePayload §1.3.2
SPEAKER_CHANGE	Diarization detects speaker switch	1	200	SpeakerPayload §1.3.3
CRM_FIELD_UPDATED	CRM webhook fires on record change	3	2000	CRMEventPayload §1.3.4
CALENDAR_EVENT	Meeting start/end/update signal	3	5000	CalendarPayload §1.3.5
PRODUCT_UPDATED	Product catalogue change ingested	4	30000	ProductPayload §1.3.6
CONFIDENCE_THRESHOLD	CRM belief drops below min_confidence	2	1000	ThresholdPayload §1.3.7
SESSION_START	Brain Session initialised	1	500	SessionPayload §1.3.8
SESSION_END	Brain Session closed or timed out	1	500	SessionPayload §1.3.8

MODULE_FAILURE	A cognitive engine reports error	1	300	FailurePayload §1.3.9
EXTERNAL_KNOWLEDGE	RAG retrieval result ready	3	1500	KnowledgePayload §1.3.10
RECOMMENDATION_ACK	Salesperson acknowledged a recommendation	2	1000	AckPayload §1.3.11
LEARNING_SIGNAL	Post-conversation outcome received	5	60000	LearningPayload §1.3.12

1.3 Payload Schemas (Selected)

1.3.1 TranscriptPayload

```
{
"text": "string — raw transcript segment text",
"speaker_id": "string — diarization label, e.g. SPEAKER_0",
"speaker_role": "enum — [SALESPERSON | BUYER | UNKNOWN]",
"start_ms": "integer — segment start time in ms from session start",
"end_ms": "integer — segment end time",
"is_final": "boolean — true if ASR has finalised this segment",
"language": "string — BCP-47 language code, e.g. en-US",
"audio_features": {
"energy_db": "float — RMS energy in decibels",
"pitch_hz": "float — mean fundamental frequency",
"speech_rate_wpm": "integer — words per minute estimate",
"pause_before_ms": "integer — silence before this segment"
}
}
```

1.3.7 ThresholdPayload

```
{
"belief_id": "string — ID of the CRM belief node",
"belief_type": "string — e.g. BUYER_INTENT | OBJECTION_ACTIVE",
"current_value": "float — current confidence score 0.0–1.0",
"threshold": "float — the crossed threshold value",
"direction": "enum — [FALLING | RISING]",
"previous_value": "float — confidence score before crossing"
}
```

Chapter 2: World Model & Cognitive Reality Model Schemas

2.1 World Model State Object

The World Model is the Brain's in-memory, continuously-updated representation of the current sales conversation. It is implemented as a versioned document, serialised to JSON for persistence checkpoints and cross-module reads.

```
{
"wm_id": "string — UUID v4",
"session_id": "string — parent session",
"version": "integer — increments on every mutation",
"updated_at_utc": "string — ISO 8601",
"buyer": {
"id": "string",
"name": "string",
"role": "string",
"company": "string",
"crm_record_id": "string"
},
"conversation_state": {
"phase": "enum — [OPENING|DISCOVERY|DEMO|OBJECTION|NEGOTIATION|CLOSING|FOLLOW_UP]",
"turn_count": "integer",
"elapsed_ms": "integer",
"last_speaker": "enum — [SALESPERSON|BUYER]"
},
"crm_beliefs": "BeliefNode[] — see §2.2",
"active_objections": "ObjectionNode[] — see §2.3",
"active_goals": "GoalNode[] — see §2.4",
"context_window": "string[] — ordered recent transcript segments",
"knowledge_refs": "KnowledgeRef[] — active RAG results",
"pending_recommendations": "Recommendation[]"
}
```

2.2 BeliefNode Schema

```
{
"belief_id": "string — UUID v4",
"belief_type": "string — from BeliefType registry",
"subject": "string — what the belief is about",
"value": "any — the believed value",
"confidence": "float — 0.0–1.0",
"evidence_ids": "string[] — event_ids that support this belief",
"competing_hypotheses": [
{ "value": "any", "confidence": "float" }
],
"created_at_utc": "string",
"updated_at_utc": "string",
"decay_rate": "float — confidence decay per minute of no reinforcement",
"is_explicit": "boolean — stated directly vs. inferred"
}
```

2.3 Recommendation Output Schema

Every output the Brain produces for the salesperson conforms to this schema. Rendering clients consume this object directly.

```
{
"rec_id": "string — UUID v4",
"session_id": "string",
"cycle_id": "string — which CDC cycle produced this",
"rec_type": "enum —
[RESPONSE_SUGGESTION|OBJECTION_HANDLE|QUESTION|ALERT|STRATEGY_SHIFT|SILENCE]",
"urgency": "enum — [IMMEDIATE|SOON|BACKGROUND]",
"content": {
"headline": "string — ≤ 12 words, shown prominently",
"body": "string — full suggested language or guidance",
"rationale": "string — why this recommendation was made (for logging)",
"alternatives": "string[] — 0–2 alternative phrasings"
},
"confidence": "float — 0.0–1.0",
"display_ttl_ms": "integer — remove from UI after N ms if not acted on",
"triggered_by": "string[] — event_ids that led to this recommendation",
"metadata": "object? — A/B test variant, model version, etc."
}
```

PART II

API Contracts, Interfaces & Communication Protocols

This part defines every internal and external API surface of the Clozflow Brain: the Session API, the Event Bus interface, cognitive engine contracts, the Recommendation delivery interface, and the integration gateway for external systems.

Chapter 3: Brain Session API

3.1 Transport & Authentication

Property	Specification
`Transport`	HTTPS/2 for REST; WSS for streaming
`Auth`	Bearer JWT — RS256 signed, 1-hour expiry, scoped per tenant
`Rate limiting`	Per tenant: 100 session starts/hour; 10,000 events/minute
`Base URL pattern`	https://brain.{tenant}.clozflow.io/v1
`Versioning`	URL-based major version; header X-CBS-Version for minor
`Idempotency`	POST endpoints accept Idempotency-Key header (UUID)

3.2 Session Lifecycle Endpoints

Method	Path	Description	Key Request Fields	Success Response
`POST`	/sessions	Create a new Brain Session	tenant_id, salesperson_id, crm_opportunity_id, metadata	201 + SessionObject
`GET`	/sessions/{id}	Retrieve session state	—	200 + SessionObject
`PATCH`	/sessions/{id}	Update session metadata	metadata, status	200 + SessionObject
`DELETE`	/sessions/{id}	Terminate session, flush memory	—	204
`GET`	/sessions/{id}/wm	Get current World Model snapshot	—	200 + WorldModel
`GET`	/sessions/{id}/recs	Get pending recommendations	since_rec_id (cursor)	200 + Recommendation[]

3.3 Event Ingestion Endpoint

Producers submit events via this endpoint. The runtime validates, envelopes, and routes them to the Event Bus. Batch ingestion is supported for non-real-time sources.

Method	Path	Max Payload	Batch Size	P99 Latency Target
`POST`	/sessions/{id}/events	64 KB	1 event	< 20 ms
`POST`	/sessions/{id}/events/batch	512 KB	100 events	< 50 ms

3.4 Recommendation Streaming Interface (WebSocket)

Real-time recommendation delivery uses WebSocket to minimise latency between Brain output and salesperson display. The client connects once per session and receives a stream of Recommendation

objects.

```
WSS /sessions/{session_id}/recommendations/stream

# Connection headers
Authorization: Bearer {jwt}
X-CBS-Version: 1

# Server → Client message envelope
{
"msg_type": "enum — [RECOMMENDATION | HEARTBEAT | SESSION_END | ERROR]",
"payload": "Recommendation | null",
"server_ts": "string — ISO 8601 server timestamp"
}

# Heartbeat interval: every 15 seconds
# Client must respond to heartbeat within 5 seconds or connection is closed
# Reconnect with exponential backoff: 100ms, 200ms, 400ms, 800ms, cap 5000ms
```

Chapter 4: Cognitive Engine Interface Contract

Every cognitive engine (Perception, Understanding, Interpretation, Prediction, Evaluation, Planning, Decision, Verification, Recommendation, Reflection) must conform to this interface. This enables the Scheduler (Vol. V §1.6) to treat all engines uniformly.

4.1 Engine Registration Schema

```
{
"engine_id": "string — unique within tenant, e.g. perception.transcript_v2",
"engine_type": "enum — [PERCEPTION|UNDERSTANDING|INTERPRETATION|PREDICTION|
EVALUATION|PLANNING|DECISION|VERIFICATION|RECOMMENDATION|REFLECTION]",
"version": "string — semver, e.g. 2.1.0",
"input_types": "string[] — EventTypes this engine consumes",
"output_types": "string[] — EventTypes this engine produces",
"latency_budget_ms": "integer — maximum allowed execution time",
"priority": "integer — scheduler priority 1–10",
"capabilities": "string[] — feature flags, e.g. [stream_aware, multi_turn]",
"health_endpoint": "string — URL for liveness check",
"config_schema": "object — JSON Schema for engine configuration"
}
```

4.2 Engine Execution Interface

Each engine exposes a single execute method. Engines are stateless across calls; all state is passed in via context.

```
# Input
{
"cycle_id": "string — Cognitive Decision Cycle identifier",
"events": "CognitiveEvent[] — events triggering this execution",
"world_model": "WorldModel — current WM snapshot",
"engine_config": "object — runtime configuration from registry",
"timeout_ms": "integer — hard deadline for this execution"
}

# Output
{
"engine_id": "string",
"cycle_id": "string",
"output_events": "CognitiveEvent[] — events produced",
"wm_mutations": "WorldModelMutation[] — atomic mutations to apply to WM",
"confidence": "float — engine confidence in its output",
"reasoning_trace": "string? — explanation for observability (optional)",
"duration_ms": "integer — actual execution time",
"status": "enum — [SUCCESS | PARTIAL | TIMEOUT | ERROR]"
}
```

4.3 World Model Mutation Contract

Engines must not directly mutate the World Model. They return a list of named mutations which the World Model Manager applies atomically after the cycle completes (Vol. II §1.9).

Mutation Type	Operation	Target	Description

UPSERT_BELIEF	Create or update	BeliefNode	Set or revise a confidence-weighted belief
RETRACT_BELIEF	Soft delete	BeliefNode	Mark belief as invalidated, retain for audit
ADD_OBJECTION	Append	ObjectionNode	Register a detected objection
RESOLVE_OBJECTION	Update status	ObjectionNode	Mark objection as resolved
UPDATE_PHASE	Set	ConversationState.phase	Advance conversation stage
PUSH_RECOMMENDATION	Append	pending_recommendations	Queue a recommendation for delivery
ADD_KNOWLEDGE_REF	Append	knowledge_refs	Attach a RAG result to the session
SET_GOAL	Upsert	GoalNode	Declare or revise a salesperson goal

PART III

Technology Stack, Infrastructure & Deployment

This part specifies the complete technology stack for building and operating the Clozflow Brain. Choices are opinionated and justified against the architectural constraints defined in Volumes V and VIII. Alternatives are noted where substitution is permissible.

Chapter 5: Recommended Technology Stack

5.1 Core Stack Overview

Layer	Technology	Rationale	Permissible Alternative
`API Gateway`	Kong Gateway (OSS)	Mature, plugin ecosystem, JWT auth built-in	AWS API Gateway, Nginx+Lua
`Session API`	Python 3.12 + FastAPI	Async-native, Pydantic schema validation, fast	Go + Gin
`Event Bus`	Apache Kafka 3.x	Ordered partitions, replay, high throughput	AWS Kinesis, Redpanda
`Cognitive Scheduler`	Python 3.12 + Celery	Distributed task queue, priority queues, retries	Temporal.io
`Engine Runtime`	Python 3.12 (async)	LLM SDK compatibility, ML library ecosystem	Rust (perf-critical engines)
`LLM Provider`	Anthropic Claude API	Long context, instruction following, low latency	OpenAI GPT-4o, Gemini 1.5
`Vector Store`	Qdrant	HNSW index, filtering, self-hostable	Pinecone, Weaviate
`World Model Store`	Redis 7.x (JSON module)	In-memory, sub-ms reads, JSON path updates	Aerospike
`Persistent Store`	PostgreSQL 16	ACID, JSONB, full-text, row-level security	CockroachDB
`Blob / Audio`	AWS S3 (or compatible)	Durable, cheap, presigned URLs for audio	GCS, Azure Blob
`Speech-to-Text`	Deepgram Nova-3	< 300 ms latency, diarization, streaming	AssemblyAI, Whisper-large-v3
`Observability`	OpenTelemetry + Grafana	Vendor-neutral traces, metrics, logs	Datadog
`Container Runtime`	Docker + Kubernetes (EKS)	Industry standard, HPA for scaling	GKE, AKS
`CI/CD`	GitHub Actions + ArgoCD	GitOps, environment parity, rollback	GitLab CI, Flux
`Secrets`	AWS Secrets Manager	Rotation, IAM integration	HashiCorp Vault

5.2 LLM Integration Specification

Each cognitive engine that delegates language tasks to an LLM must follow this integration contract. Engines are responsible for prompt construction; the LLM is responsible only for language generation.

Parameter	Value / Constraint
`Model`	claude-sonnet-4-6 (production); claude-haiku-4-5 for latency-critical paths
`Max context tokens`	180,000 tokens (leave 4,096 for output)
`Temperature`	0.3 for recommendation generation; 0.0 for classification tasks

`Max output tokens`	1,024 for recommendations; 256 for classification; 4,096 for summaries
`System prompt`	Injected per engine — see §5.3 for template
`Retry policy`	3 retries, exponential backoff: 200ms, 400ms, 800ms
`Timeout`	Hard: 4,000 ms for real-time engines; 15,000 ms for background engines
`Streaming`	Enabled for Recommendation engine only — stream tokens to UI as generated
`Cost control`	Budget alert at $0.10/session; hard stop at $0.50/session

5.3 LLM System Prompt Template

Every engine injects this base system prompt, then appends its engine-specific instructions. The World Model snapshot is serialised into the user turn, not the system prompt, to maximise cache hits.

```
You are a component of the Clozflow Cognitive Brain, a real-time sales intelligence
system. Your role is: {ENGINE_ROLE}.

Constraints:
- Respond only in valid JSON matching the output schema provided.
- Do not fabricate facts about the buyer, product, or conversation.
- Express uncertainty using confidence scores, not hedged language.
- Keep recommendations ≤ 60 words unless producing a detailed explanation.
- Never recommend actions that require information not present in the World Model.

Output schema: {ENGINE_OUTPUT_SCHEMA}

Current timestamp: {ISO8601_TIMESTAMP}
Conversation phase: {PHASE}
Active objections: {OBJECTION_COUNT}
```

Chapter 6: Infrastructure Architecture

6.1 Service Topology

The Clozflow Brain is deployed as a set of independently scalable microservices. The following diagram describes the service topology and data flow between services.

```
┌─────────────────────────────────────┐
│ API GATEWAY │
│ Kong · JWT auth · Rate limiting │
└─────────────────────────────────────┘
│ HTTPS/2
┌──────────────────────────────────────────────┐
▼ ▼ ▼
┌────────────────┐ ┌────────────────┐ ┌────────────────┐
│ Session API │ │ Event API │ │ Stream API │
│ FastAPI │ │ FastAPI │ │ FastAPI/WSS │
└────────────────┘ └────────────────┘ └────────────────┘
│ Redis │ Kafka │ Redis
▼ ▼ ▼
┌──────────────────────────────────────────────────────────────────┐
│ EVENT BUS (Kafka) │
│ Topics: cognitive.events cognitive.recommendations│
└──────────────────────────────────────────────────────────────────┘
│
┌──────────────▼─────────────────┐
│ COGNITIVE SCHEDULER │
│ Celery + Priority Queue │
└──────────────────────────────┘
│ Dispatches to engine workers
┌──────────────────────────────────────────────────────────────────┐
▼ ▼ ▼ ▼ ▼ ▼ ▼
Perc. Underst. Interp. Predict. Evaluate. Plan. Recommend.
Worker Worker Worker Worker Worker Worker Worker
│ │ │ │ │ │ │
┌──────────────────────────────────────────────────────────────────┐
│ WM mutations
┌──────────────▼─────────────────┐
│ WORLD MODEL MANAGER │
│ Redis (hot) + Postgres │
└──────────────────────────────┘
```

6.2 Kubernetes Deployment Specifications

Service	Replicas (min/max)	CPU Request/Limit	Memory Request/Limit	HPA Trigger
`session-api`	2 / 10	0.5 / 2.0	512Mi / 1Gi	CPU > 60%
`event-api`	2 / 20	0.5 / 2.0	256Mi / 512Mi	CPU > 70%
`stream-api`	2 / 10	0.25 / 1.0	256Mi / 512Mi	Connections > 500
`cognitive-scheduler`	2 / 4	1.0 / 4.0	1Gi / 2Gi	Queue depth > 1000

perception-worker	3 / 15	1.0 / 4.0	1Gi / 4Gi	CPU > 65%
recommendation-worker	2 / 10	1.0 / 4.0	1Gi / 4Gi	CPU > 65%
wm-manager	2 / 6	0.5 / 2.0	512Mi / 2Gi	CPU > 60%
kafka (broker)	3 (fixed)	2.0 / 4.0	4Gi / 8Gi	Manual
redis	3 (cluster)	1.0 / 2.0	2Gi / 4Gi	Manual

PART IV

Latency Budgets, Performance Contracts & SLAs

Real-time sales assistance is only valuable if it arrives before the conversation moves on. This part defines the hard latency budgets for every processing stage, the performance contracts for each service, and the observability hooks required to enforce them.

Chapter 7: End-to-End Latency Budget

7.1 Target: Speech to Recommendation Display

The total budget from speech audio arriving to a recommendation appearing in the salesperson's UI is 1,500 ms at P95. The following table breaks this into stage budgets.

Stage	Component	P50 Target	P95 Target	P99 Limit	Action if Exceeded
`1. STT`	Deepgram streaming	120 ms	280 ms	400 ms	Fallback to Whisper, alert
`2. Event Creation`	event-api ingest	5 ms	15 ms	30 ms	Drop event, circuit break
`3. Event Bus Transit`	Kafka produce + consume	10 ms	25 ms	50 ms	Reduce batch size
`4. Scheduler Dispatch`	Celery task queue	5 ms	20 ms	40 ms	Add worker replicas (HPA)
`5. Perception Engine`	Engine execute	50 ms	100 ms	180 ms	Partial output + continue
`6. Understanding Engine`	Engine execute	60 ms	120 ms	200 ms	Partial output + continue
`7. Recommendation Engine`	LLM call + format	400 ms	750 ms	1,200 ms	Use cached template
`8. WM Mutation Apply`	Redis write	3 ms	10 ms	20 ms	Queue mutation, proceed
`9. WebSocket Delivery`	stream-api push	5 ms	15 ms	30 ms	Reconnect client
`TOTAL`	Speech → UI display	658 ms	1,335 ms	2,150 ms	Degrade gracefully

7.2 Background Engine Latency Budgets

Background engines (Reflection, Learning, Knowledge Indexing) operate on relaxed latency budgets. They must complete before the next conversation turn ends, not before the current recommendation is needed.

Engine	Trigger	P95 Budget	Blocking?
`Interpretation Engine`	Per transcript segment	300 ms	No — enriches WM asynchronously
`Prediction Engine`	Every 3rd turn or phase change	500 ms	No — feeds next cycle
`Evaluation Engine`	After Prediction completes	400 ms	No
`Planning Engine`	After Evaluation completes	300 ms	No
`Decision Engine`	After Planning completes	150 ms	No
`Verification Engine`	After Decision, before Recommend	100 ms	Yes — gates Recommendation
`Reflection Engine`	Post-session, async	30,000 ms	No — batch job

`Learning Indexer`	Post-session, async	60,000 ms	No — batch job

7.3 Confidence Thresholds

Confidence values govern when the Brain acts, escalates, or abstains. The following table specifies all confidence thresholds that appear as configurable parameters in the system.

Parameter	Default Value	Effect When Crossed	Config Key
`Minimum belief confidence to act`	0.55	Belief is treated as probable; enables planning	brain.belief.act_threshold
`Minimum recommendation confidence`	0.70	Recommendation is delivered to UI	brain.rec.min_confidence
`Objection detection threshold`	0.65	ObjectionNode created in World Model	brain.objection.detect_threshold
`Phase transition confidence`	0.75	Conversation phase advances to next stage	brain.phase.transition_threshold
`Competing hypothesis trigger`	0.20	Two beliefs within 0.20 of each other → both maintained	brain.belief.competing_delta
`Belief decay floor`	0.10	Beliefs below this are retracted automatically	brain.belief.decay_floor
`Recommendation deduplication window`	0.85	Suppress similar rec if cosine sim > threshold	brain.rec.dedup_similarity
`STT confidence minimum`	0.80	Segments below this marked uncertain, engine adjusts	brain.stt.min_confidence

PART V

Testing Framework, Validation & Quality Gates

No cognitive system may be deployed to production without passing the quality gates defined in this part. Tests are organized into five tiers, each targeting a different layer of the architecture.

Chapter 8: Five-Tier Testing Architecture

8.1 Tier Overview

Tier	Name	Scope	Tooling	Must Pass Before
1	Unit Tests	Individual functions, schema validators, mutation logic	pytest, pydantic	Every commit
2	Engine Contract Tests	Each engine's input/output contract, latency, error handling	pytest + mock LLM	Every PR merge
3	Integration Tests	Multi-engine CDC cycles end-to-end on synthetic sessions	pytest + testcontainers	Every deployment
4	Conversation Simulations	Full conversations with scripted buyer personas, correctness	Custom sim harness	Weekly + pre-release
5	Load & Chaos Tests	SLA validation under production load, failure injection	k6, Chaos Mesh	Every major release

8.2 Engine Contract Test Specification

Every cognitive engine must pass all of the following contract tests before being registered in the engine registry. Tests are run with a mock LLM that returns deterministic outputs.

Test ID	Test Name	Pass Condition
ECT-01	Schema validation — input	Engine rejects malformed input with status=ERROR within 10 ms
ECT-02	Schema validation — output	Engine output validates against output schema in 100% of runs
ECT-03	Latency budget	P95 execution time < latency_budget_ms declared at registration
ECT-04	Timeout handling	Engine returns status=TIMEOUT before deadline; no hanging threads
ECT-05	Idempotency	Same input + cycle_id produces identical wm_mutations
ECT-06	Statelessness	Engine produces same output regardless of call order in session
ECT-07	Confidence range	Output confidence always in [0.0, 1.0]
ECT-08	Empty event handling	Engine handles empty events list without error; returns status=SUCCESS
ECT-09	World Model mutation validity	All mutations are of registered types; targets exist in WM schema
ECT-10	Reasoning trace format	reasoning_trace is a string ≤ 2,000 characters when present

8.3 Conversation Simulation Framework

The simulation harness replays scripted conversations against a live Brain instance and evaluates recommendation quality, timing, and World Model correctness.

Minimum Simulation Corpus (required for release)

- 50 DISCOVERY phase conversations — varying industries, product types
- 30 OBJECTION handling conversations — price, timing, authority objections
- 20 NEGOTIATION conversations — discount requests, multi-stakeholder
- 20 CLOSING conversations — soft close, hard close, lost deal
- 10 EDGE CASES — silent buyer, technical failure mid-call, language switch

Metric	Threshold	Measurement Method
`Recommendation relevance`	> 80% rated relevant by human evaluators	Blind evaluation panel, 3 raters
`Recommendation latency P95`	< 1,500 ms from speech end to UI display	Simulation harness timing
`World Model accuracy`	> 90% beliefs match ground truth at session end	Scripted truth labels
`Phase detection accuracy`	> 88% correct phase classification	Ground truth phase labels
`Objection detection recall`	> 85% of labelled objections detected	Labelled simulation scripts
`False positive objections`	< 10% of non-objection turns flagged	Labelled simulation scripts
`Recommendation dedup rate`	< 5% duplicate recommendations per session	Automatic dedup metric
`Session crash rate`	< 0.1% of simulated sessions	Error log analysis

Chapter 9: Observability & Quality Gates

9.1 Required Telemetry Instrumentation

Every service and engine must emit the following signals using OpenTelemetry. Dashboards in Grafana are generated automatically from these signals.

Signal Type	Signal Name	Labels Required
`Counter`	brain_events_total	event_type, session_id, tenant_id, status
`Histogram`	brain_engine_duration_ms	engine_id, engine_type, status, cycle_id
`Histogram`	brain_recommendation_latency_ms	rec_type, urgency, tenant_id
`Gauge`	brain_world_model_belief_count	session_id, belief_type
`Gauge`	brain_active_sessions	tenant_id, environment
`Counter`	brain_llm_tokens_total	engine_id, model, direction (in/out)
`Counter`	brain_llm_errors_total	engine_id, error_type
`Histogram`	brain_wm_mutation_apply_ms	mutation_type, status
`Counter`	brain_recommendations_delivered	rec_type, urgency, tenant_id
`Counter`	brain_recommendations_acknowledged	rec_type, tenant_id

9.2 Production Quality Gate Checklist

The following checklist must be signed off by the engineering lead before any version is promoted to production.

- All Tier 1–3 tests pass with 0 failures
- Tier 4 simulation corpus passes all threshold metrics in §8.3
- P95 end-to-end recommendation latency < 1,500 ms under 2× expected peak load
- Memory leak test: 8-hour sustained load shows < 5% memory growth
- All required telemetry signals are emitting in staging environment
- Kafka consumer lag < 500 messages under peak load
- Redis hit rate > 90% for World Model reads
- All secrets rotated and confirmed in Secrets Manager
- Rollback procedure tested and documented in runbook
- On-call rotation confirmed and runbook reviewed by on-call engineer

PART VI

Build Onboarding Sequence — Step-by-Step Implementation Guide

This part provides the authoritative build sequence for any engineering team or AI system implementing the Clozflow Brain from scratch. Following this sequence minimises integration risk by ensuring each layer is validated before the next is added.

Chapter 10: Phased Build Sequence

Phase 1 — Foundation (Weeks 1–2)

1.1 Infrastructure provisioning

- Provision Kubernetes cluster (3 nodes minimum, 16 vCPU / 64 GB each)
- Deploy Kafka (3 brokers), Redis Cluster (3 nodes), PostgreSQL (primary + read replica)
- Configure Kong API Gateway with JWT plugin and rate limiting
- Set up GitHub Actions CI pipeline with automated test runs
- Configure OpenTelemetry collector → Grafana stack

1.2 Core schemas & validation

- Implement and publish all schemas from Part I as Pydantic models
- Write schema validation tests (Tier 1) — target 100% coverage
- Publish schemas to shared Python package: clozflow-schemas v1.0
- Implement EventType registry as a versioned configuration file

1.3 Kafka topic provisioning

- Create topic: cognitive.events — 12 partitions, retention 24h
- Create topic: cognitive.recommendations — 6 partitions, retention 2h
- Create topic: cognitive.learning — 3 partitions, retention 30d
- Configure consumer groups per engine type

Phase 2 — Session & Event Layer (Weeks 3–4)

2.1 Session API

- Implement all Session Lifecycle Endpoints (§3.2) in FastAPI
- Implement JWT authentication middleware
- Write World Model initialisation on session creation
- Store session metadata in PostgreSQL; hot state in Redis
- Run integration tests (Tier 2) against Session API

2.2 Event API & Bus

- Implement event ingest endpoints (§3.3) with Pydantic validation
- Implement Kafka producer with idempotent writes
- Implement batch ingest endpoint with transaction support
- Write Event Bus consumer base class (all engines inherit this)

2.3 World Model Manager

- Implement WorldModel as Redis JSON document with versioning
- Implement atomic mutation application (§4.3 mutation types)
- Implement PostgreSQL checkpoint persistence every 60 seconds
- Implement WM snapshot endpoint for engine consumption

Phase 3 — Cognitive Scheduler (Week 5)

3.1 Scheduler implementation

- Implement Celery task queue with 5 priority levels (matching EventType priorities)
- Implement engine registry (load from configuration, validate on startup)
- Implement engine selection logic based on incoming event types
- Implement execution deadline enforcement (timeout_ms per engine)
- Implement failure handling: mark failed, apply fallback, continue cycle
- Write Tier 2 contract tests for the Scheduler itself

Phase 4 — Cognitive Engines (Weeks 6–10)

4.1 Build engines in dependency order

- Week 6: Perception Engine — transcript ingestion, STT integration, speaker diarization
- Week 7: Understanding Engine — intent classification, entity extraction, sentiment
- Week 8: Interpretation Engine — why-analysis, hidden objection detection, emotion modelling
- Week 9: Prediction Engine, Evaluation Engine, Planning Engine (can parallelize)
- Week 10: Decision Engine, Verification Engine, Recommendation Engine, Reflection Engine

4.2 Engine validation per engine

- Run all 10 Engine Contract Tests (ECT-01 through ECT-10) before integration
- Achieve P95 latency ≤ declared budget in isolation tests
- Integrate one engine at a time; run end-to-end smoke test after each

Phase 5 — Recommendation Delivery (Week 11)

5.1 WebSocket stream & UI integration

- Implement WebSocket stream endpoint (§3.4) in FastAPI
- Implement heartbeat and reconnection logic
- Implement recommendation deduplication (cosine similarity, sliding window)
- Implement display TTL expiry signalling
- Build minimal reference UI (React) consuming the WebSocket stream
- Run end-to-end latency measurement — must achieve P95 < 1,500 ms

Phase 6 — Knowledge & Memory (Week 12)

6.1 RAG pipeline

- Deploy Qdrant vector store with product catalogue, playbooks, objection library
- Implement embedding pipeline (text-embedding-3-small or equivalent)
- Implement retrieval engine: query on conversation context, top-5 results
- Attach results as KnowledgeRef objects in World Model

6.2 Long-term memory

- Implement post-session summarisation in Reflection Engine

- Store buyer profiles, historical objections, past outcomes in PostgreSQL
- Implement memory retrieval injection into World Model at session start

Phase 7 — Validation & Release (Weeks 13–14)

7.1 Full test suite

- Run Tier 3 integration tests — full CDC cycles on synthetic sessions
- Run Tier 4 simulation corpus (minimum 130 conversations per §8.3)
- Run Tier 5 load test at 2× peak load — validate all latency SLAs
- Run Tier 5 chaos test — kill one Kafka broker, one worker; verify graceful degradation

7.2 Production readiness checklist

- Complete all items in the Production Quality Gate Checklist (§9.2)
- Conduct security review: pen test API endpoints, validate JWT expiry, audit IAM roles
- Write and review operational runbook with on-call team
- Deploy to staging with production-equivalent load for 48 hours
- Promote to production with feature flag — 5% traffic → 25% → 100%

PART VII

Enterprise Integration — Concrete Implementation Specifications

Volume VII defined the Enterprise Cognitive Architecture at a conceptual level. This part provides the concrete integration specifications for the three highest-priority enterprise system connections: CRM, Calendar, and Product Catalogue.

Chapter 11: CRM Integration

11.1 Webhook Contract

CRM systems push updates to the Brain via webhook. The Brain does not poll CRM systems. The following contract applies to all supported CRM platforms (Salesforce, HubSpot, Pipedrive).

Field	Type	Description
`event_type`	string	CRM-specific event, e.g. opportunity.updated
`crm_platform`	enum	SALESFORCE | HUBSPOT | PIPEDRIVE | CUSTOM
`record_type`	enum	OPPORTUNITY | CONTACT | ACCOUNT | ACTIVITY
`record_id`	string	CRM-native record identifier
`session_id`	string	Clozflow session to route this event to
`changed_fields`	object	Map of field_name → {old_value, new_value}
`timestamp_utc`	string	ISO 8601 — when the CRM change occurred
`signature`	string	HMAC-SHA256 of payload body, key from Secrets Manager

11.2 Salesforce Specific Implementation

- Use Salesforce Platform Events (not Outbound Messaging) for < 500 ms delivery
- Subscribe to: OpportunityChangeEvent, ContactChangeEvent, ActivityChangeEvent
- Map Salesforce Stage field to Clozflow ConversationPhase using configurable mapping table
- Store Salesforce OAuth 2.0 tokens per tenant in Secrets Manager, refresh 10 min before expiry
- Rate limit: Salesforce allows 1,000 Platform Events/hour on Enterprise tier — monitor and back-off

Chapter 12: Speech-to-Text Pipeline Implementation

12.1 Deepgram Streaming Integration

```
# Connection parameters for Deepgram Nova-3 streaming
wss://api.deepgram.com/v1/listen
?model=nova-3
&smart;_format=true
&diarize;=true
&diarize;_version=3
&utterances;=true
&interim;_results=true
&endpointing;=300
&vad;_events=true
&language;=en-US

# Audio format: PCM 16-bit, 16kHz sample rate, mono
# Send chunks every 100ms for optimal latency
# On UtteranceEnd event → emit TRANSCRIPT_SEGMENT cognitive event (is_final=true)
# On interim result → emit TRANSCRIPT_SEGMENT (is_final=false) for early processing
```

12.2 Fallback Strategy

Condition	Fallback Action	Recovery
`Deepgram latency > 400ms P95`	Switch to Whisper-large-v3 on local GPU node	Auto-switch back after 5 min stable
`Deepgram API error (5xx)`	Buffer audio, retry after 200ms × 3	Switch to fallback after 3 failures
`STT confidence < 0.80`	Emit event with is_uncertain=true flag	Engines apply wider confidence intervals
`Network disconnect`	Buffer up to 10s of audio in memory	Reconnect + replay buffer on recovery

PART VIII

Security Implementation Specifications

This part defines the concrete security implementations required at each layer of the Brain. Security controls must be present before any production deployment.

Chapter 13: Security Controls Specification

13.1 Authentication & Authorisation

Control	Implementation	Validation Test
`API Authentication`	RS256 JWT, 1-hour expiry, issued by Clozflow Auth Service	Request with expired token returns 401
`Tenant Isolation`	Every DB query filters by tenant_id; enforced in ORM layer	Cross-tenant query attempt returns 403
`Engine-to-Engine Auth`	mTLS between internal services via Istio service mesh	Non-mTLS request rejected at sidecar
`Kafka Auth`	SASL/SCRAM-SHA-512, per-topic ACLs per service account	Wrong credentials → connection refused
`Redis Auth`	requirepass + TLS in transit; Redis ACL per service	Unauthenticated connection rejected
`Secret Access`	IAM role per service, least privilege, no hardcoded secrets	Audit log shows no secrets in env vars

13.2 Data Privacy Requirements

- Audio data must never be stored beyond the session — delete within 60 seconds of session end
- Transcript text is PII — encrypt at rest (AES-256) in PostgreSQL using column-level encryption
- World Model snapshots are tenant-scoped — never cross tenant_id boundaries
- Recommendation logs retained for 90 days for audit purposes, then purged
- GDPR/CCPA: implement DELETE /tenants/{id}/data endpoint that purges all records within 30 days
- All data transfers between services must use TLS 1.3 minimum

PART IX

Complete Configuration Reference

This part provides the complete, annotated configuration file for the Clozflow Brain runtime. All configurable parameters from all prior volumes are consolidated here.

Chapter 14: Brain Runtime Configuration

14.1 brain.yaml — Complete Configuration Reference

```
# Clozflow Brain Runtime Configuration
# Version: 1.0 | brain.yaml

session:
max_duration_minutes: 120
idle_timeout_minutes: 10
max_concurrent_per_tenant: 500
checkpoint_interval_seconds: 60

belief:
act_threshold: 0.55 # Minimum confidence to act on a belief
decay_floor: 0.10 # Beliefs below this are retracted
competing_delta: 0.20 # Gap that triggers competing hypothesis mode
decay_rate_per_minute: 0.02 # Default decay for unconfirmed beliefs

recommendation:
min_confidence: 0.70 # Below this, suppress recommendation
dedup_similarity: 0.85 # Cosine sim threshold for deduplication
dedup_window_turns: 3 # Look back N turns for duplicates
max_pending: 5 # Max queued recs before oldest is dropped
default_display_ttl_ms: 8000 # Remove from UI if not acted on

objection:
detect_threshold: 0.65
auto_resolve_turns: 5 # If no reinforcement in N turns, resolve

phase:
transition_threshold: 0.75
allowed_transitions: # Explicit FSM — no arbitrary jumps
OPENING: [DISCOVERY]
DISCOVERY: [DEMO, OBJECTION, CLOSING]
DEMO: [OBJECTION, NEGOTIATION, CLOSING]
OBJECTION: [DISCOVERY, DEMO, NEGOTIATION, CLOSING]
NEGOTIATION: [CLOSING, OBJECTION]
CLOSING: [FOLLOW_UP]
FOLLOW_UP: []

llm:
default_model: claude-sonnet-4-6
fast_model: claude-haiku-4-5 # Used for latency-critical classification
temperature_recommendation: 0.3
temperature_classification: 0.0
max_output_tokens_recommendation: 1024
max_output_tokens_classification: 256
timeout_realtime_ms: 4000
timeout_background_ms: 15000
budget_alert_per_session_usd: 0.10
budget_hard_stop_per_session_usd: 0.50
retry_attempts: 3
retry_backoff_ms: [200, 400, 800]
```

```
stt:
provider: deepgram
model: nova-3
min_confidence: 0.80
fallback_provider: whisper
fallback_trigger_latency_ms: 400
audio_format: pcm_16bit_16khz_mono
chunk_interval_ms: 100
endpointing_ms: 300

knowledge:
vector_store: qdrant
embedding_model: text-embedding-3-small
top_k_retrieval: 5
similarity_threshold: 0.70
max_context_chars: 4000

scheduler:
worker_concurrency: 8 # Celery workers per pod
priority_levels: 5
task_timeout_ms: 5000 # Hard deadline for any scheduled task
max_queue_depth: 10000

observability:
otlp_endpoint: http://otel-collector:4317
trace_sample_rate: 0.10 # Sample 10% of traces in production
slow_request_threshold_ms: 1500
heartbeat_interval_ms: 15000
```

Appendices

A Cross-Volume Dependency Map

The following table shows which sections of Volume X implement the architectural intent from each prior volume.

Prior Volume	Architectural Intent	Implemented in V10
`Vol. I — Philosophy`	Event-driven, non-prompt activation; cognitive hierarchy	§3.3 Event API; §7.3 Confidence Thresholds; §14.1 phase FSM
`Vol. II — Execution`	Cognitive Execution Cycle (CEC); event lifecycle	Part II API Contracts; §5 Scheduler; §8.2 contract tests ECT-05/06
`Vol. III — Reality Model`	CRM as structured, confidence-weighted, versioned document	§2.1 World Model Schema; §2.2 BeliefNode; §7.3 confidence params
`Vol. IV — Intelligence`	CDC phases: Perceive→Reflect; iterative reasoning	§4.1 Engine Interface; §10 Phase 4 build sequence
`Vol. V — Runtime (COS)`	Kernel, Scheduler, Memory Manager, Communication Layer	§6 Infrastructure; §14.1 scheduler config; Kafka + Redis spec
`Vol. VI — Extensibility`	Cognitive capability registration and governance	§4.1 Engine Registration Schema; engine registry config
`Vol. VII — Enterprise`	CRM/calendar integration, application gateway	Part VII §11–12; CRM webhook contract
`Vol. VIII— Engineering`	Runtime sessions, execution scheduling, observability	§6.2 K8s specs; §9.1 telemetry; §13 security
`Vol. IX — Assessment`	Evidence collection, cognitive evaluation, benchmarks	§8 Testing Architecture; §8.3 simulation metrics; §9.2 quality gate

B Glossary of Implementation Terms

Term	Definition
`CDC`	Cognitive Decision Cycle — the canonical 10-phase reasoning process (Vol. IV)
`CEC`	Cognitive Execution Cycle — the event-driven execution loop (Vol. II)
`CRM`	Cognitive Reality Model — the Brain's internal world representation (Vol. III)
`COS`	Cognitive Operating System — the runtime infrastructure layer (Vol. V)
`WM`	World Model — the live, in-session, mutable state document (§2.1)
`Engine`	A stateless cognitive processing unit implementing the Engine Interface (§4)
`BeliefNode`	A single confidence-weighted belief in the World Model (§2.2)
`Mutation`	An atomic, typed change instruction applied to the World Model (§4.3)
`Brain Session`	A bounded cognitive context for one sales conversation (§3.2)
`Recommendation`	The Brain's output object delivered to the salesperson UI (§2.3)
`Confidence`	A float [0.0, 1.0] representing the Brain's certainty about a belief or output
`TTL`	Time-to-live — the duration before an event or recommendation expires

RAG	Retrieval-Augmented Generation — knowledge retrieval from vector store (§12, §6.1)
HPA	Horizontal Pod Autoscaler — Kubernetes scaling mechanism (§6.2)
mTLS	Mutual TLS — bidirectional certificate authentication between services (§13.1)
OTLP	OpenTelemetry Protocol — telemetry export format (§9.1)

C Minimum Viable Brain — Reduced Scope for Prototype

For teams building an initial prototype before full implementation, the following components form the Minimum Viable Brain (MVB). The MVB delivers the core value — real-time transcript analysis and recommendation — without the full enterprise stack.

Minimum Viable Brain Components

- Session API (single-tenant, no multi-tenant isolation required for prototype)
- Event API + Kafka (single broker acceptable for prototype)
- World Model in Redis only (no PostgreSQL checkpoint for prototype)
- Perception Engine + Understanding Engine + Recommendation Engine (3 engines only)
- WebSocket recommendation delivery (no deduplication required for prototype)
- Deepgram STT integration (no fallback required for prototype)
- Single LLM model (no fast/slow split required for prototype)

The MVB excludes: enterprise CRM integration, multi-tenancy, long-term memory, RAG knowledge, all background engines (Interpretation through Reflection), chaos resilience, and security controls. These must all be implemented before any production deployment involving real customer data.

D Document Completion Confirmation

This volume completes the Clozflow Cognitive Brain Specification. An engineering team or AI system in possession of all ten volumes has the full specification required to build, validate, deploy, and operate the Clozflow Cognitive Brain.

Volume	Title	Status
I	Philosophy & Design Principles	Complete
II	Cognitive Execution Architecture	Complete
III	Reality Representation Architecture	Complete
IV	Cognitive Intelligence Architecture	Complete
V	Cognitive Runtime & Systems Architecture	Complete
VI	Cognitive Development & Extensibility Architecture	Complete
VII	Enterprise Cognitive Architecture	Complete
VIII	Enterprise Cognitive Engineering	Complete
IX	Enterprise Cognitive Assessment & Maturity	Complete
X	Implementation Engineering Specification	Complete — this volume

END OF SPECIFICATION

CLOZFLOW COGNITIVE BRAIN SPECIFICATION · VOLUME X · v1.0